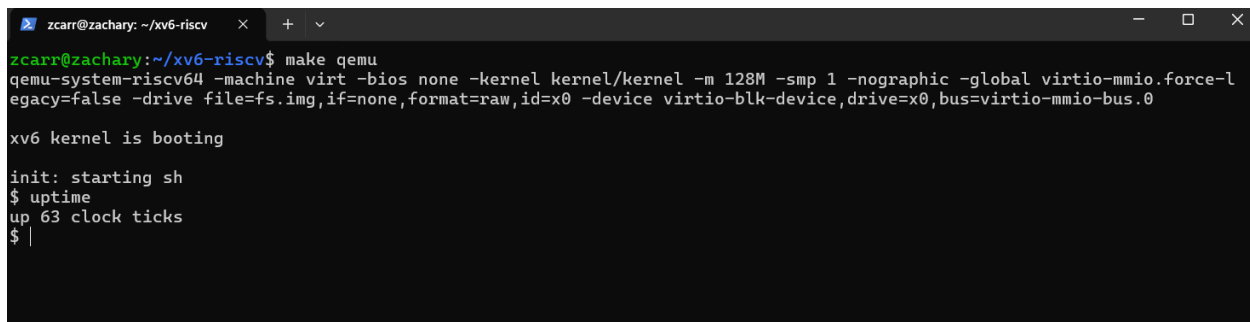


PA 1: Implementing xv6 User Commands and System Calls

Please replace red text with your report text and any tables or figures, names of any accompanying files, etc. Remember to commit all the files for your lab submission, to put the URL for your private xv6 repo in the Teams assignment, to submit the Teams assignment, and to give the instructor and TA access to your repo.

Task 1. (5 pts) Implement the uptime utility

I implemented the uptime utility in user/uptime.c. The program calls the existing xv6 uptime() system call and prints the number of clock ticks since the system started. I also added the program to the UPROGS section of the Makefile so that it is compiled and available from the xv6 shell. After building xv6 with make qemu, I ran the command and received:



```
zcarr@zachary: ~/xv6-riscv
zcarr@zachary:~/xv6-riscv$ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 1 -nographic -global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

init: starting sh
$ uptime
up 63 clock ticks
$ |
```

This confirmed that the uptime utility was working correctly.

Task 2. (10 pts) Implement a time1 command that reports elapsed time.

I created user/time1.c in the pa1-syscalls branch of my xv6 repository. The time1 command accepts another command and its arguments, creates a child process using fork(), and then uses exec() in the child to run the requested program. The parent process calls wait() until the child finishes. I used the uptime() system call before creating the child and again after the child finished so I could calculate the total elapsed time in clock ticks. I also added time1 to the UPROGS section of the Makefile so that it is compiled and available from the xv6 shell. This matches the requirements for Task 2.

I tested the command with no arguments, with matmul, with two concurrent matmul processes, and with sleep 10. My results were:

```
xv6 kernel is booting

init: starting sh
$ time1
usage: time1 command [args ...]
$ time1 matmul
Time: 6 ticks
elapsed time: 6 ticks
$ time 1 sleep 10
exec time failed
$ time1 sleep 10
elapsed time: 10 ticks
$ time1 matmul & time1 matmul &
time1 sleep 10$
Time: 11 ticks
elapsed time: 12 ticks
Time: 10 ticks
elapsed time: 11 ticks
elapsed time: 11 ticks
$ |
```

Task 3. (10 pts) Keep track of how much cputime a process has used.

```
zcarr@zachary:~/xv6-riscv$ grep -n "cputime" kernel/proc.h kernel/proc.c kernel/trap.c
kernel/proc.h:91:  uint cputime;
kernel/proc.c:127:  p->cputime = 0;
kernel/trap.c:88:    p->cputime++;
kernel/trap.c:161:    myproc()->cputime++;
```

I modified the xv6 process structure so the kernel can track CPU time for each process. I added a cputime field to struct proc in kernel/proc.h. I initialized this field to zero in allocproc() in kernel/proc.c whenever a new process is created. I also modified kernel/trap.c so that the current process's cputime is incremented on timer interrupts before the process yields the CPU. After making these changes, I rebuilt xv6 successfully and verified that the system booted normally.

Task 4. (10 pts) Implement a wait2() system call that waits for a child to exit and returns the child's status and rusage.

I implemented a new wait2() system call that waits for a child process to exit and returns both the child's exit status and resource usage information. The resource usage is stored in a struct rusage, which currently contains the child process's CPU time. This matches the assignment requirement that wait2() return status and rusage, with cputime as the current resource field.

I changed the following files:

kernel/pstat.h — I created this file and defined struct rusage with a uint cputime field.

user/user.h — I added the forward declaration for struct rusage and the user-level prototype int wait2(int *, struct rusage *);.

user/usys.pl — I added an entry for wait2() so the user-level assembly stub is generated.

kernel/syscall.h — I added a new system call number for SYS_wait2.

kernel/syscall.c — I added the declaration for sys_wait2() and connected SYS_wait2 to the new system call handler.

kernel/sysproc.c — I implemented sys_wait2(), which reads the two user arguments and passes them to the kernel-level wait2() function.

kernel/defs.h — I added the kernel prototype for wait2() so it can be called outside proc.c.

kernel/proc.c — I included pstat.h and implemented wait2() by adapting the existing kwait() logic. The new function waits for a zombie child, copies the child's exit status to user space, copies the child's cputime into a struct rusage, frees the child process, and returns the child PID.

After making these changes, I rebuilt xv6 with make clean and make qemu. The kernel compiled successfully and xv6 booted to the shell prompt, confirming that the new wait2() system call was integrated without build or boot errors.

Task 5. (10 pts) Implement a time command that runs the command given to it as an argument and outputs elapsed time, CPU time, and %CPU used.

I implemented user/time.c. The program accepts a command and its arguments, records the starting time using uptime(), and creates a child process using fork(). The child executes the requested command using exec(). The parent calls the new wait2() system call to wait for the child and obtain its CPU time through struct rusage. After the child finishes, the program calculates elapsed time and CPU percentage using the child's CPU time divided by the elapsed time. I added the time command to the UPROGS section of the Makefile and tested it using the provided matmul and sleep programs.

```
xv6 kernel is booting

init: starting sh
$ time matmul
Time: 5 ticks
elapsed time: 5 ticks, cpu time: 5 ticks, 100% CPU
$ time sleep 10
elapsed time: 10 ticks, cpu time: 0 ticks, 0% CPU
$ time matmul & time matmul &
$ Time: 9 ticks
elapsed time: 10 ticks, cpu time: 5 ticks, 50% CPU
Time: 8 ticks
elapsed time: 9 ticks, cpu time: 5 ticks, 55% CPU
|
```

The first test shows that matmul is CPU-intensive and used approximately 100% of the CPU when running by itself. The sleep 10 test shows an elapsed time of 10 ticks but a CPU time of 0 ticks because the process spends most of its time sleeping instead of executing. When two matmul processes were run at the same time, each process received approximately half of the

available CPU time. These results show that the time command correctly reports elapsed time, CPU time, and CPU utilization.

Task 6. (5 pts) Summarize what you learned from doing this assignment. Describe any difficulties you faced and if/how you overcame them.

From this lab, I learned more about how system calls work in xv6 and how user programs communicate with the kernel. I learned how to add a new system call by making changes across several files, including the system call table, user-level declarations, kernel functions, and process structures. I also learned how xv6 keeps track of processes and how CPU time can be measured using timer interrupts. Creating the wait2() system call helped me understand how information such as a process's exit status and CPU usage can be passed from the kernel back to a user program. The time command also showed me the difference between elapsed time and actual CPU time. For example, matmul used close to 100% CPU when running alone, while sleep 10 had an elapsed time of 10 ticks but used 0 CPU ticks.

I encountered several difficulties while completing the assignment. One problem was making sure changes were placed in the correct files and structures. For example, I initially placed the cputime field in the wrong structure and had to move it into struct proc. I also had some Makefile formatting problems and a duplicate wait entry in usys.pl, which caused compilation errors. Another challenge was making sure the new wait2() system call was connected correctly through all the different parts of xv6. I overcame these problems by reading the compiler errors, checking the affected files, and testing after each change instead of making all of the changes at once. By the end, xv6 compiled successfully, and my tests with matmul, sleep, and two simultaneous matmul processes showed that the CPU-time tracking and time command were working correctly.

Extra Credit (5 points). Discuss limitations of our time command. (Hint: For one limitation, consider what would happen if the command that is being timed forks child processes).

One limitation of our time command is that it only reports the CPU time of the direct child process that it creates and waits for using wait2(). If the command being timed creates its own child processes using fork(), the CPU time used by those additional processes is not included in the result. For example, if the original process uses very little CPU itself but creates several children that perform CPU-intensive work, our time command could report a low CPU time even though the entire program used much more CPU.

Another limitation is that CPU usage is measured in clock ticks, so the measurement is not very precise for programs that finish quickly. A short program could execute between timer interrupts and report zero or very few CPU ticks. The %CPU calculation also uses integer arithmetic, which causes rounding and makes the percentage less precise. A more complete implementation

could track the CPU time of a process and all of its descendants and use a higher-resolution timing method to provide more accurate results.